Ders 14 Genişletilmiş Notu: Tavsiye Sistemleri, İşbirlikçi Filtreleme ve Dönem Değerlendirmesi

Ders 14 Genişletilmiş Notu: Tavsiye Sistemleri, İşbirlikçi Filtreleme ve Dönem Değerlendirmesi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dönemin Son Dersi ve Tavsiye Sistemlerinin Yeri
- 2. Tavsiye Sistemi Nedir?
- 3. İçerik Tabanlı Tavsiye Sistemleri
- 4. İşbirlikçi Filtrelemenin Fikri
- 5. Bellek Tabanlı ve Model Tabanlı İşbirlikçi Filtreleme
- 6. Kullanıcı Tabanlı Filtreleme
- 7. Ürün Tabanlı Filtreleme
- 8. İşbirlikçi Filtrelemenin Sorunları
- 9. Model Tabanlı Yöntemler ve Matris Ayrıştırma
- 10. Tavsiye Sistemlerinin Bugünü
- 11. Makine Öğrenmesinin Bugünü ve Kapanış Tavsiyeleri
- 12. Ödev Soruları Üzerinden Akademik Notlar

1. Dönemin Son Dersi ve Tavsiye Sistemlerinin Yeri

Bu ders dönemin son dersidir. Hoca, öğrencilerin ödev ve sınav yoğunluğunu bildiği için kısa bir ders yapacağını söyleyerek başladı. Bir önceki derste işbirlikçi filtrelere (collaborative filtering) giriş yapılmıştı; bu ders o konunun devamıdır.

Hocaya göre makine öğrenmesinin klasik ders programlarında tavsiye sistemleri (recommender systems) yoktu. Son yıllarda ise makine öğrenmesinin en başarılı uygulama alanlarından biri oldu. İnternet üzerinden hizmet veren pek çok firma, müşterilerine ürün önermek ve satışı artırmak için bu sistemleri kullanıyor. Bu yaklaşımı ilk kullanıp faydasını gösteren ve yayılmasını sağlayan öncü kuruluş Netflix'tir. Bugün Amazon, YouTube, Pinterest, Google gibi milyonlarca müşteriye hizmet veren kuruluşlar da tavsiye sistemlerinden yararlanıyor. Hoca bu yüzden son 3-4 yıldır tavsiye sistemlerini derste anlatıyor.

2. Tavsiye Sistemi Nedir?

Tavsiye sistemlerini herkes internet alışverişinde ya da Netflix'te film izlerken görüyor: sistem kullanıcıya bir şeyler öneriyor.

En basit tavsiye, çok satan ya da o an popüler olan ürünleri göstermektir. Ör. "şu anda popüler olan kitaplar şunlardır" demek. Bu, çoğunluğun beğendiği ürünlerden haberdar olmayı sağlar. Ama bu genel bir tavsiyedir.

Asıl önemli olan **kişiselleştirilmiş tavsiyedir**. Kişinin zevkine uygun öneri gelirse o ürünü alma olasılığı artar. Çalışmalar da bunun üzerine yoğunlaşıyor: "Ne yaparsak müşterilerin satın alma miktarını artırırız?"

Hoca tavsiye sistemlerini iki ana yaklaşım altında topladı: içerik tabanlı sistemler ve işbirlikçi filtreler.

3. İçerik Tabanlı Tavsiye Sistemleri

İçerik tabanlı (content-based) tavsiye sistemleri genel olarak makine öğrenmesinin temel, klasik yöntemlerini kullanır ve kişilerin özelliklerine bakar.

- **Kişi özelliklerine göre öneri:** "Üniversite mezunu, belli bir yaşı geçmiş, spor yapan, sanatla ilgilenen kişilere ne önerebilirim?" sorusuyla çalışır. Özellikleri (mezun olunan okul, yaş, yapılan sporlar vb.) size benzeyen kişilerin seçimleri önerilir.
- **Önceki seçimlerin türüne göre öneri:** Kişi sosyal içerikli filmler izliyorsa sosyal içerikli filmler, romantik filmler izliyorsa romantik filmler önerilir.

İlk tavsiye sistemleri böyleydi. Hocaya göre bu yaklaşımda "aşırı bir kişiselleştirme yoktur": sistem kişiyi bir özellik grubuna ya da bir film türüne indirger.

4. İşbirlikçi Filtrelemenin Fikri

İşbirlikçi filtreleme, hocaya göre şu anda en çok kullanılan yaklaşımdır. Çıkış noktası, insanların zevklerinin kaba kategorilere indirgenemeyeceğidir.

Hoca bunu film zevki üzerinden anlattı. Bir kişi komedi filmleri izliyor olabilir ama her komediyi izlemez. Ör. Cem Yılmaz türü komediler hoşuna gidiyor, ama Recep İvedik türü filmler ilgisini çekmiyor. Üstelik bu kişi yalnız komedi de izlemiyor; biyografi ve dram da izliyor. Biyografilerde bile bir alt tercihi olabilir: daha çok sanatçıların ya da siyasetçilerin biyografilerini izliyor olabilir. Yani seçimlerimiz kaba değildir, alt seçimlerimiz vardır. Kişiyi "komedi izliyor" diye tek bir şeye indirgemek doğru değildir.

İşbirlikçi filtrelerin hedefi bu alt seçimleri yakalamaktır. İşbirlikçi filtre ana başlığa bakıp "komedi izliyor, komedi göstereyim" demez. Kişinin daha önce seçtiği ürünlere (item) bakar ve bu seçimleri diğer kişilerin seçimleriyle eşleştirir. Bunu iki yoldan yapabilir:

- Kişiye **benzeyen kişilerin** yaptığı seçimlere göre öneride bulunmak.
- Kişinin beğendiği ürünlere **benzeyen ürünlere** göre öneride bulunmak. Ör. Cem Yılmaz türü komedilere yüksek puan verdiyseniz bunlara benzeyen bir filmi size önermek.

5. Bellek Tabanlı ve Model Tabanlı İşbirlikçi Filtreleme

Hoca işbirlikçi filtreleri iki gruba ayırdı:

- **Bellek tabanlı (memory-based):** Yalnız kullanıcı ve ürün bilgilerini kullanır. Bütün olay iki taraf etrafında döner: kullanıcılar (user) ve ürünler (item). Netflix için kullanıcılar izleyiciler, ürünler filmlerdir; Amazon için kullanıcılar müşteriler, ürünler satılan mallardır. İki türü vardır:
  - Kullanıcı tabanlı filtreleme (user-item filtering, user-based): kişiye benzeyen kullanıcıların seçimlerine göre öneri.
  - Ürün tabanlı filtreleme (item-item filtering, item-based): bir ürüne benzeyen ürünlere göre öneri.
- **Model tabanlı (model-based):** Makine öğrenmesi yöntemleriyle bir model kurar ve öneriyi modele göre yapar (Bölüm 9).

6. Kullanıcı Tabanlı Filtreleme

6.1 Puan matrisi

Örnek puan üzerinden yürütüldü. Netflix'te kullanıcıların izledikleri filmlere 1 ile 5 arasında puan (rating) verdiğini varsayalım; 5 en yüksek, 1 en düşük puan.

Bu puanlar bir matriste tutulur:

- **Satırlar** kullanıcılardır (derste örnek adlar: Ali, Ayşe, Can).
- **Sütunlar** ürünlerdir (Movie 1, Movie 2, Movie 3, Movie 4…).

Herkes her filme puan vermez. Ör. Ali ikinci filmi izleyip 5, dördüncü filmi izleyip 4 vermiş olabilir; diğer hücreler boştur.

Problem şudur: Ali'ye izlemediği filmler arasından, en çok beğenebileceğini düşündüğümüz filmi önermek istiyoruz. Netflix'in "daha önce şunu izlediğine göre bunlar ilgini çekebilir" demesi de budur.

6.2 Benzerlik: Pearson korelasyon katsayısı

Kullanıcı tabanlı filtrelemenin ilk işi, kişiye benzeyen kullanıcıları bulmaktır. Hoca, benzerliğin bir önceki derste görülen kovaryans konusuyla ilgili olduğunu hatırlattı. Benzerlik için iki ölçü kullanılır: **Pearson korelasyon katsayısı** (Pearson correlation coefficient) ve **kosinüs benzerliği** (cosine similarity).

r_a ve r_u, A ve U kullanıcılarının M ürüne verdiği puanlar olsun. $\bar{r}_a$ ve $\bar{r}_u$ bu kullanıcıların ortalama puanlarıdır. İki kullanıcının puanlarının kovaryansı:

$$\text{cov}(a,u) = \frac{1}{M}\sum_{i=1}^{M}(r_{a,i} - \bar{r}_a)(r_{u,i} - \bar{r}_u)$$

Ortalama da M puanın toplamının M'ye bölünmesiyle bulunur. Pearson korelasyon katsayısı, kovaryansın iki kullanıcının standart sapmalarının çarpımına bölünmesidir:

$$\text{sim}(a,u) = \frac{\text{cov}(a,u)}{\sigma_a\,\sigma_u} = \frac{\sum_i (r_{a,i} - \bar{r}_a)(r_{u,i} - \bar{r}_u)}{\sqrt{\sum_i (r_{a,i} - \bar{r}_a)^2}\,\sqrt{\sum_i (r_{u,i} - \bar{r}_u)^2}}$$

Benzerlik değerleri -1 ile $+1$ arasında değişir.

6.3 Yalnız ortak puanlanan ürünler kullanılır

Benzerlik ölçülürken dikkat edilmesi gereken kural: **iki kullanıcının da puanladığı ürünler** kullanılmalıdır. Hoca bunu bir örnekle gösterdi.

Örnek 1: Ortak ürün kuralı.

	Movie 1	Movie 2	Movie 3	Movie 4
A	4	2		5
B		5		3

- Verilen: A, Movie 1'e 4, Movie 2'ye 2, Movie 4'e 5 vermiş. B, Movie 2'ye 5, Movie 4'e 3 vermiş.
- B birinci filmi izlememiştir. Bu yüzden A-B benzerliği yalnız iki kullanıcının da puanladığı Movie 2 ve Movie 4 üzerinden hesaplanır.
- Sonuç (ara adım eklendi): Ortak ürünlerde A'nın puanları $(2, 5)$, B'nin puanları $(5, 3)$. A birinde düşük, diğerinde yüksek; B tam tersi. Ortalamalar ortak ürünler üzerinden alınırsa $\text{sim}(A,B) = -1$ çıkar. Ortalamalar her kullanıcının bütün puanları üzerinden alınırsa ($\bar{r}_A = 11/3 \approx 3{,}67$, $\bar{r}_B = 4$) sonuç $\approx -0{,}99$ olur. İki durumda da A ile B'nin zevkleri birbirine ters görünür.

  Ek bilgi (derste anlatılmadı): Ortak puanlanan ürün sayısı çok azsa Pearson katsayısı güvenilmez olur. Yukarıdaki örnekte olduğu gibi yalnız iki ortak ürünle katsayı kolayca ± 1'e gider. Uygulamada bu yüzden ortak ürün sayısı az olan benzerlikler küçültülür ya da hiç kullanılmaz.

A kişisinin kime benzediğini bulmak için A'nın, puan matrisindeki **bütün kullanıcılara** benzerliği bu şekilde tek tek ölçülür.

6.4 En benzer K kullanıcı

Benzerlikler ölçüldükten sonra genellikle **K en yakın komşu** (K-nearest neighbor) mantığı kullanılır. Benzerlikler büyükten küçüğe sıralanır ve A'ya en çok benzeyen K kişi seçilir. Amaç, A'ya benzeyen kişilerin beğendiği şeyleri A'ya önermektir.

6.5 Puan tahmini

Şimdi A kullanıcısının izlemediği bir p ürününe (ör. Movie 3) kaç puan vereceğini tahmin etmek istiyoruz. Bunun için:

1. A'ya en çok benzeyen K kişi bulunur.
2. Bu kişilerden p ürününü **izlemiş olanlar** kullanılır. Tahmin yapabilmek için kişinin hem ilk K'ye girmiş olması hem de o filmi izlemiş olması gerekir.
3. Tahmin şu formülle hesaplanır:

$$p_{a,p} = \bar{r}_a + \frac{\sum_{b \in N} \text{sim}(a,b)\,(r_{b,p} - \bar{r}_b)}{\sum_{b \in N} |\text{sim}(a,b)|}$$

Burada N, p ürününü puanlamış en benzer K kullanıcıdır. Formülün okunuşu: A'nın kendi ortalama puanından başlanır. Her benzer kullanıcı b için, b'nin o filme verdiği puanın b'nin kendi ortalamasından farkı alınır ve A ile b'nin benzerliğiyle ağırlıklandırılır. Toplam, normalize etmek için benzerliklerin toplamına bölünür.

Hoca örneği şöyle kurdu: $K = 3$ için A'ya en çok benzeyen 3 kişi bulunur. Ör. B ilk 3'e girmiş ve üçüncü filme 4 vermiş; C de ilk 3'e girmiş ve üçüncü filme bir puan vermiş. Bu kişilerin üçüncü filme verdikleri puanlara göre A'nın üçüncü filme vereceği puan tahmin edilir.

6.6 Öneri seçimi

Tahminler A'nın izlemediği her film için hesaplandıktan sonra, tahmini puanı en yüksek olanlar önerilir.

Örnek 2: Öneri seçimi.

- Verilen tahminler: Movie 3 için 4,2; Movie 7 için 4,9; Movie 12 için 2,8.
- İşlem: Tahminler sıralanır: Movie 7 (4,9) > Movie 3 (4,2) > Movie 12 (2,8).
- Sonuç: Önce Movie 7 önerilir. Belli bir eşiğin üstündekiler önerilecekse Movie 3 de önerilebilir. Movie 12 önerilmez.

6.7 Neden "bellek tabanlı"?

Bu yöntem bellek tabanlıdır, çünkü yalnız puan matrisine bakar. Kişinin diğer özelliklerini ya da filmin özelliklerini (içeriği, festival filmi olup olmadığı vb.) hiç kullanmaz. Tek yaptığı, puan matrisini kullanıp size benzeyen kullanıcıların seçimlerine göre öneri yapmaktır. Hocaya göre burada makine öğrenmesi tabanlı bir tahmin yoktur; yalnız puanların matematiksel benzerliğine göre bir öneri vardır.

7. Ürün Tabanlı Filtreleme

7.1 Fikir

İkinci yaklaşım ürün tabanlı (item-based) filtrelemedir. Buradaki mantık, **ürünler arasındaki benzerliği** bulmaktır. Puan matrisi yine aynıdır: satırlarda kullanıcılar, sütunlarda filmler vardır.

Hoca tablete bir örnek matris yazdı ve şu soruyu sordu: A kişisi Movie 4'e kaç puan verir? Ürün tabanlı yaklaşımda bunun için Movie 4'e en çok benzeyen ürünler bulunur. Hocanın örneğinde Movie 1 ile Movie 4 birbirine benziyordu. Bu durumda A'nın Movie 1'e verdiği puandan yararlanarak Movie 4 için tahmin yapılabilir.

7.2 Neden daha çok kullanılır?

Hocaya göre ürün tabanlı yaklaşım, kullanıcı tabanlıdan daha çok kullanılır; Netflix de bu mantığı kullanıyor. Sistem izlediğiniz filmlere bakar ve "şu filmi izlediğine göre büyük ihtimalle şu filmden de hoşlanırsın" der. Milyonlarca kişi arasından size benzeyen kişileri bulup onların izlediklerine göre öneri yapmak yerine, sizin izlediğiniz ürünlere benzeyen ürünler aranır.

7.3 Kosinüs benzerliği

Ürün tabanlı işbirlikçi filtrelemede en çok kullanılan ve başarılı olan benzerlik ölçüsü **kosinüs benzerliğidir**. Kosinüs benzerliği iki vektör arasındaki açıyı ölçer: açı ne kadar küçükse iki vektör birbirine o kadar yakındır.

U, hem a hem b ürününü puanlayan kullanıcılar olsun. $\bar{r}_u$, u kullanıcısının ortalama puanıdır. Ürünlerin benzerliği, her puandan kullanıcının ortalaması çıkarılarak normalize edilmiş şekilde hesaplanır:

$$\text{sim}(a,b) = \frac{\sum_{u \in U}(r_{u,a} - \bar{r}_u)(r_{u,b} - \bar{r}_u)}{\sqrt{\sum_{u \in U}(r_{u,a} - \bar{r}_u)^2}\,\sqrt{\sum_{u \in U}(r_{u,b} - \bar{r}_u)^2}}$$

Hoca anlatırken önce "U ürünün ortalaması" dedi, sonra "U kullanıcılarının ortalaması" diye düzeltti: çıkarılan, kullanıcının ortalamasıdır.

> **Ek bilgi (derste anlatılmadı):** Kullanıcı ortalaması çıkarılarak hesaplanan bu ölçü literatürde "düzeltilmiş kosinüs benzerliği" (adjusted cosine similarity) diye geçer. Ortalamanın çıkarılması, hep yüksek puan veren ve hep düşük puan veren kullanıcıların farkını giderir. Ürün tabanlı tahmin de genelde, kullanıcının puanladığı ürünlerin hedef ürüne benzerlikleriyle ağırlıklandırılmış ortalaması olarak hesaplanır.

Hoca burada da önemli olanın **en çok benzeyen ürünler** üzerinden ilerlemek olduğunu vurguladı. Bütün ürünler için benzerlik ölçülür, ama tahmin en benzer K ürün üzerinden yapılır.

8. İşbirlikçi Filtrelemenin Sorunları

Hoca bu bölümü öğrencilere sorarak işledi: "Kullanıcı tabanlı ya da ürün tabanlı yöntemin sizce problem yaşayacağı noktalar ne olabilir?"

8.1 Ölçek ve hız

Soru: Film ve kullanıcı sayısı çok fazla olunca yöntem yavaş çalışabilir mi? **Cevap:** Hoca bunu doğru buldu ve birinci problem olarak ele aldı. Netflix'i düşünün: milyonlarca kullanıcı ve on binlerce film (diziler, filmler…) vardır. Amazon'da on binlerce ürün (çarşaftan kitaba, kaleme) ve dünya çapında milyonlarca, belki milyarlarca kullanıcı vardır. Matris çok büyüktür.

Kullanıcı tabanlı yöntemde A'nın kime benzediğini bulmak için milyonlarca kişiyle karşılaştırma yapmak gerekir. Bu son derece yavaştır. Hocaya göre kullanıcı tabanlı yöntem bu yüzden gerçek dünyada kullanılamayan bir yöntemdir.

8.2 Seyreklik

İkinci sorun matrisin boşluğudur, yani **seyreklik** (sparsity). Bir kişi on binlerce filmin ancak birkaç yüzünü izler; Amazon'da bir kullanıcı 50-100 farklı ürün alır, hatta genelde 10 farklı ürün alır. Matris çok büyük ama büyük ölçüde boştur. Herkesin ortak puanladığı ürünleri bulmaya çalıştığınızda çok fazla boşlukla karşılaşırsınız.

Hem kullanıcı tabanlı hem ürün tabanlı yaklaşım hem matrisin büyüklüğünden hem de boşlukların fazlalığından sıkıntı çeker.

8.3 Soğuk başlangıç

Soru: Başka ne problem olabilir? (Öğrenci: soğuk başlangıç.) **Cevap:** Hoca kesinlikle doğru olduğunu söyledi. **Soğuk başlangıç** (cold start) problemi iki yönlüdür:

- Yeni bir ürünü kime önereceğim?
- Yeni bir kullanıcıya ne önereceğim?

Yeni kullanıcıya benzeyen kişileri bulabilmek için önce o kullanıcının bir şeyler izlemesi, bir şeyler alması gerekir. Yani önce bir süre veri biriktirmek gerekir.

8.4 Hep aynı önerileri yapmak

Soru: Süreç içinde sistemin sürekli aynı önerilerde bulunması bir problem olabilir mi? Bu "sömürü ve keşif" (exploitation-exploration) diye ifade ediliyor. **Cevap:** Hoca bunun da doğru bir problem olduğunu söyledi.

8.5 Benzerliklerin birbirine yaklaşması

Soru: Kullanıcı ve film sayısı çok artınca, benzerlikleri -1 ile 1 arasına sıkıştırdığımız için birbirine çok yakın değerler çıkabilir mi? Çok sayıda kişi birbirine benzer görünebilir mi? **Cevap:** Hoca bunun çok kötü olmayabileceğini söyledi: birbirine benzeyen kullanıcı sayısı fazlaysa sistem zaten onların seçtiğini önerir. Ona göre asıl sorun, kişi her önüne geleni izlemeye başlarsa zevkine uygunluğun karışmasıdır. Ama puan verildiği sürece bu da büyük bir sorun olmaz.

Hoca bu noktada Netflix'in başta puan aldığını, şimdi ise yalnız beğendim/beğenmedim (like/dislike) aldığını hatırlattı. Bunun daha kesin bir bilgi olduğunu, ama öneri başarısını düşürüp düşürmediğini bilmediğini söyledi.

8.6 Az bilinen ürünler

Kıyıda köşede kalmış bir ürünü oylayan kimse olmayabilir. Hocaya göre bu, "hep aynı şeyleri öneriyor" probleminin mantığıyla kesişir: hep aynı şeyler önerildiği için bazı ürünler hiç dikkat çekmez. Ör. çok sanatsal bir film var; izleseniz çok beğeneceksiniz, ama kimse uğraşıp izlemiyor. Kimse izlemeyince kimseye önerilmiyor ve film köşede kalıyor. Hoca bunu farklı bir soğuk başlangıç olarak gördü: film uzun zamandır sistemde duruyor, ama izlenmediği için fark edilmiyor.

8.7 Sıra dışı zevkler

Çok özel zevki olan kişilere öneri yapmak zor olabilir. Ör. felsefe kitabı okuyan biri, felsefe okuyanların bile seçmeyeceği kitapları seçiyor olabilir; bu zevke benzeyen çok kişi yoktur. Hoca bunu çok büyük bir problem olarak görmediğini söyledi: kullanıcı sayısı çok arttığında, çok nadir de olsa mutlaka birine benzeyen biri çıkar.

8.8 Ortak hesap

Hocanın asıl sıkıntı olarak gördüğü nokta ortak hesap kullanımıdır. Netflix'te kullanıcı profili tanımlatılmasının sebebi, o kullanıcının zevkini anlamaktır. Ebeveynlerin izledikleriyle gençlerin izledikleri aynı tür şeyler değildir. Ama insanlar genelde tembellikten profil değiştirmez. O zaman aynı profilin içinde aksiyon filmleri, komedi filmleri ve başka türler karışır; 3-4 farklı kişinin zevki bir arada olur. Bu, değerlendirmeyi zorlaştıran bir özelliktir.

Hoca bu sorunların, bellek tabanlı işbirlikçi filtrelerin genel problemleri olduğunu söyleyerek bölümü kapattı.

9. Model Tabanlı Yöntemler ve Matris Ayrıştırma

9.1 Fikir

İşbirlikçi filtrelerin ikinci türü model tabanlı filtrelerdir. Buradaki mantık: yalnız kullanıcıların puan bilgisini değerlendirerek öneri yapmak yerine, başka özellikleri de dikkate alarak öneri

yapmak gerekir. Model tabanlı filtreler makine öğrenmesi yöntemleriyle bir model kurar ve ürünü bu modele göre önerir.

Model tabanlı sistemler genellikle önce **boyut azaltır**. Bunun için PCA (temel bileşen analizi), tekil değer ayrışımı (Singular Value Decomposition, SVD), sinir ağı (neural network) ve matris ayrıştırma (matrix factorization) gibi yöntemler kullanılır. Böylece az önce konuşulan seyreklik problemi çözülmeye çalışılır.

9.2 Model tabanlı yaklaşımların türleri

- **Kümeleme tabanlı (clustering-based).**
- **Matris ayrıştırma tabanlı:** SVD ya da olasılıksal matris ayrıştırma (probabilistic matrix factorization) kullanılır.
- **Derin öğrenme tabanlı:** Çok katmanlı sinir ağları (multi-layer neural network) kullanılır. Hocaya göre şu anda en popüler olanıdır, çünkü başarısı yüksektir.

Hoca kümeleme tabanlı yöntem için şunu söyledi: "Aslında cluster oluşturmakla ilgisi yok." Burada da en benzer K örnek kullanılır, yani aslında K en yakın komşu mantığıdır. "Kümeleme tabanlı" denmesinin sebebi, kümelerin benzer örneklerden oluşmasıdır. (Bkz. Ders Sonrası Notlar.)

9.3 Matris ayrıştırma

Hoca, çok kullanılan yaklaşımlardan birinin, tavsiye sisteminin başarısını artıracak yöntemi bulana ödül veren bir yarışmada önerildiğini söyledi. (Yarışmanın adı için bkz. Ders Sonrası Notlar.)

Elimizde N kullanıcı ve M filmden oluşan bir puan matrisi R olsun. Ürün sayısı çok fazla olduğu için R çok seyrektir. Matris ayrıştırma R'yi iki matrisin çarpımına ayırır:

```
       R (N x M, seyrek)   ~   U (N x D, dolu)   x   V (D x M, dolu)
satır: kullanıcı           satır: kullanıcı       sütun: film
```

$$R \approx U\,V, \qquad U \in \mathbb{R}^{N\times D}, \quad V \in \mathbb{R}^{D\times M}$$

- U matrisinde yine N satır vardır; her satır bir kullanıcıdır. Ama satırlar azaltılmış D boyutundadır ve matris tamamen doludur, hiç boşluk yoktur.
- V matrisi D satırdan oluşur; her sütunu bir filmi gösterir. Bu matris de tamamen doludur.
- U ile V çarpılınca orijinal R matrisi elde edilir. Aradaki fark: R seyrekken U ve V tamamen doludur.

Bu ayrıştırma genelde SVD gibi yöntemlerle yapılır. Hoca bunu bir önceki derste görülen temel bileşen analizine (PCA) bağladı. İşlem ana matristen üretilmiş **azaltılmış özelliklerle** yapılır. Böylece seyreklik problemi giderilmiş olur ve yalnız çıkarılmış önemli bilgi kullanılır.

> **Ek bilgi (derste anlatılmadı):** UV çarpımı tamamen dolu olduğu için, R'de boş olan bir hücrenin (kullanıcının izlemediği filmin) değeri de çarpımdan okunabilir. Tahmini puan, o kullanıcının U'daki satırı ile o filmin V'deki sütununun iç çarpımıdır. U ve V genelde yalnız R'nin dolu hücrelerindeki hatayı en aza indirecek şekilde öğrenilir.

10. Tavsiye Sistemlerinin Bugünü

Hocaya göre tavsiye sistemleri son yıllarda çok önemli bir konu ve bu konuda çok çalışma yapılıyor. Bugün her şey internet ve e-ticaret (e-commerce) üzerinden yürüyor. Satış olması da gerekmiyor: Spotify ve YouTube gibi platformlar da kimin neyi izlediğini biliyor, çünkü reklamları ve önerileri buna göre yapıyor. YouTube'da müzik dinlerken ya da video izlerken gelen öneriler hep bu makine öğrenmesi yöntemleriyle yapılıyor.

Buradaki hedef kullanıcıya yardım etmektir. YouTube'da milyarlarca video vardır; kullanıcının her şeyi karıştırıp aradığını kendi başına bulması mümkün değildir. Sistem izlediklerine ve önceki seçimlerine bakarak yeni çıkan içerikleri ona önerir. Büyük bilgi içinden en uygununu bulmak kişi için çok zor olduğu için tavsiye sistemleri popüler bir konudur.

Hoca bir de kendi görüşünü ekledi. Sistemlerin arada, kişiye pek uygun olmayan ürünleri de "azıcık azıcık" önerdiğini düşünüyor. Aksi halde bir ürünün popülerliği artırılamaz. Bir ürün en çok satan olmasa bile kullanıcı ona yönlendirildiğinde, çok seçici değilse onu tercih edebilir.

11. Makine Öğrenmesinin Bugünü ve Kapanış Tavsiyeleri

Dersin son akademik bölümünde hoca, uzun yıllardır okuduğu ve takip ettiği araştırmalara dayanarak makine öğrenmesinin nereye gittiğini ve nelere dikkat edilmesi gerektiğini anlattı.

11.1 İnsan gibi davranan sistemlerden büyük veriye

Başlangıçta makine öğrenmesi, yapay zekânın bir alt başlığı olarak, problemlerin insan gibi öğrenerek çözülmesine odaklanıyordu: yüz tanıma, parmak izi tanıma, insansız taşıt, söyleneni anlama (doğal dil işleme). Bunlar insan gibi davranan sistemler için yapılan çalışmalardı.

Bugün ise olay büyük veriye (big data) kaymış durumda. Ortada çok büyük bir veri var ve bu veriden bilgi çıkarmak çok zor. Bu yüzden makine öğrenmesi yöntemleri ağırlıklı olarak burada kullanılıyor. Tavsiye sistemleri de böyledir: milyonlarca ürünü kişiye nasıl göstereceksiniz? Netflix'e her gün onlarca film konuyor. Kişinin kendine uygun olanı bulmasına yardım ederseniz platformunuz daha çok tercih edilir.

Örnek: Saldırı tespit sistemleri. Eskiden saldırı tespit sistemleri dizgi eşleme (string matching) tabanlıydı. Bilinen saldırılar algoritmayla tespit edilebiliyordu. Ama saldırgan, siz onu tespit edince farklı bir saldırıyla karşınıza çıkabiliyor. Bu yüzden bugün yapılması gereken, normale uymayan, normale benzemeyen durumu yakalamaya çalışmaktır: **anomali tespiti** (anomaly detection).

11.2 İstatistikten derin öğrenmeye

İlk yöntemler tamamen matematiğe ve istatistiğe dayalıydı. Şimdi yavaş yavaş derin öğrenmeye doğru gidiliyor. Yine ağır matematik ve istatistik var, ama "0,8'den büyükse şöyle, 0,7 ise böyle" gibi kurallar yerine daha insan gibi davranan, daha sezgisel çözümlere gidilmeye çalışılıyor. Belki başarıların yükselmesinin sebebi de budur.

> **Hocanın vurgusu:** Derin öğrenme her şeyin çözümü değildir.

- Derin öğrenme için çok fazla örnek gerekir. Ör. bir hastalık teşhisi için yalnız 100 hasta verisi topladıysanız derin öğrenme iyi bir seçim değildir.
- Veriyi GAN (Generative Adversarial Network) ile çoğaltmak da hocaya göre çok etkili değildir.
- Hoca, klasik makine öğrenmesi yöntemleriyle elde edilen sonuçları, makine öğrenmesinin bir alt kolu olan derin öğrenmeyle elde edilenlerle her zaman karşılaştırmayı tavsiye etti. Hangisi daha etkiliyse o kullanılmalıdır.
- Pek çok yerde bir destek vektör makinesiyle (SVM) ya da en basit bir karar ağacıyla (decision tree) elde edilen sonuç, derin öğrenmeyle elde edilenden daha iyi çıkabiliyor.

11.3 Lisansüstü öğrencilere tavsiyeler

> **Hocanın vurgusu:** Lisansüstü öğrenci olarak Python'da hazır kütüphaneleri "Lego gibi" birleştirmeye takılmayın.

- Asıl problemlere odaklanın. Ör. "soğuk başlangıç problemini burada çözmek için ne yapabilirim?" ya da "matris ayrıştırma iyi bir şey; seyreklik problemini çözmek için ne yapabilirdim?" gibi sorular.
- "Bir veri kümem var; A yöntemini uyguladım başarı %80, B yöntemini uyguladım %90" türü çalışmalar 15-20 yıl önce kaldı. Bunlar ilk zamanlarda yapılan şeylerdi. Şimdi uygulamalardaki problemleri çözmeye yönelmek gerekiyor.
- Veriyi alma aşamasından başlayarak yapılacak iyileştirmelerle fark yaratmaya çalışın. Ör. derin öğrenme özellik seçimini kendisi yapıyor; ama "derin öğrenmeye vermeden önce şöyle bir ön işlem yapsam başarı nasıl artar?" diye araştırılabilir. Hoca bunu "bir taraftan inek verdim, öbür taraftan hamburger çıktı" benzetmesiyle anlattı: kara kutuya veri verip sonucu almak yerine, müdahalelerinizin neyi değiştirdiğine bakın.

- Müdahale edin, ama modele yanlılık (bias) katmadan, onu şartlandırmadan.
- Her zaman iyileştirmek gerekmez. “Şöyle bir şey önerdim, denedim ama olmadı” demek de bir sonuçtur. Diğer araştırmacılara o yöne gitmemeleri gerektiğini gösterir.

Hocaya göre bugün herkes makine öğrenmesi yapıyor ve çoğunun tek yaptığı bir veri kümesine bir yöntem uygulamak. Oysa olay bu değil; çok daha kapsamlı ve yapılması gereken çok iş var.

12. Ödev Soruları Üzerinden Akademik Notlar

Dersin sonundaki soru-cevap bölümünde ödevle ilgili sorular üzerinden bazı akademik noktalar konuşuldu. Not, tarih ve sınavla ilgili kısımlar bu nota alınmamıştır.

12.1 Basit bir kullanıcı tabanlı tavsiye sistemi

Hoca son ödevde çok basit, kullanıcı tabanlı bir tavsiye sistemi istedi. Kullanıcı tabanlı yöntemin Netflix gibi gerçek sistemlerde kullanılmadığını, ama böyle bir sistemin temelde nasıl çalıştığını görmek için verdiğini söyledi. Varyans gibi basit hesapların (ortalamayı hesaplayıp çıkarmak gibi) kendiniz yapılmasını istedi.

12.2 Kategorik özellikler ve one-hot kodlama

Soru: Spotify şarkılarıyla k-means ödevinde, şarkı adı ve ID silindikten sonra kategorik özellik olarak yalnız sanatçı adı kalıyor. Bu da silinirse kategorik özellik kalmıyor gibi. **Cevap:** Hoca önce genel kuralı verdi: bir özellik her örnekte farklıysa (uniqueness oluşturuyorsa) kullanılmamalıdır. Şarkı adı ve şarkının ID’si bu yüzden kullanılamaz. Ama sanatçı adı unique değildir; bir sanatçının birden fazla şarkısı vardır. Bu yüzden sanatçı adı bir özellik olabilir.

Veri kümesinde yaklaşık 34 bin farklı sanatçı vardı. One-hot kodlama (one-hot encoding) ile bu özellik 34 bin yeni özellik demektir. Hocaya göre özellik sayısı bu kadar artınca bu genelde başarıyı düşürür. Önerisi: iki yolu da deneyin. Özelliği ekleyip başarıya bakın, özelliği çıkarıp başarıya bakın ve sonucu raporda tartışın.

Müzik anahtarı (key: sol, la gibi) da kategorik bir özelliktir ve kullanılabilir.

> **Hocanın vurgusu:** Spotify gerçekten kümeleme kullanıyor: şarkıları kümelere ayırıyor ve öneriyi kümeler üzerinden yapıyor.

12.3 Dirsek yöntemi ve “inertia”

Soru: Dirsek yönteminde (elbow method) k-means’in hazır “inertia” özelliği kullanılabilir mi, yoksa hata kendimiz mi hesaplanmalı? **Cevap:** Inertia, örneklerin küme merkezlerine uzaklıklarının kareleri toplamıdır (küme içi kare hata toplamı). Hoca, hazır değerin kullanılmasının kendisi için fark etmediğini söyledi. Önemli olan dönüm noktasını doğru bulmaktır.

> **Hocanın vurgusu:** Dirsek yönteminde hedef en düşük hatayı bulmak değil, dönüm noktasını bulmaktır.

K artırıldıkça hata hep düşer. Her örnek ayrı bir kümeye yerleşirse her örneğin küme merkezine uzaklığı sıfır olur ve hata sıfıra iner. İstenen bu değildir. Hata önce hızla düşer, bir noktadan sonra yavaş yavaş düşmeye başlar; o dönüş noktası seçilir. Bunu sayılara bakarak da görebilirsiniz, ama dirsek grafiğini çizdirince dönüş daha net görülür.

Soru: Hata önce 10’dan 5’e hızlı iniyor, sonra 5’ten 3’e, 3’ten 2,5’e, 2,5’ten 2’ye ve 2,4’e iniyor. Hangi noktayı almalıyız? **Cevap:** Hoca görmeden kesin bir şey söyleyemeyeceğini, ama karar verirken bu düşüşün yavaşladığı yere bakılması gerektiğini söyledi; dönüm noktası en iyi sonucu verir.

Hoca k-means’in en basit yöntemlerden biri olduğunu, ama bugün çok kullanıldığını hatırlattı. Ör. önce veriyi kümelere ayırıp sonra kümelerin sonucuna göre etiketleme yapılan işlerde kullanılır. Bu işlerde kaç gruba ayıracağınızı genelde bilmezsiniz. Dirsek yöntemini anlamak bu yüzden önemlidir.

Dersin Toparlaması

Bu son derste tavsiye sistemleri, makine öğrenmesinin güncel ve başarılı bir uygulama alanı olarak ele alındı. Önce içerik tabanlı sistemler ile işbirlikçi filtreler ayrıldı: içerik tabanlı sistemler kişinin ya da ürünün özelliklerine bakarken, işbirlikçi filtreler kullanıcıların puanlarından alt zevkleri yakalamaya çalışır. Bellek tabanlı işbirlikçi filtrelemenin iki türü görüldü. Kullanıcı tabanlı yöntemde Pearson benzerliğiyle en benzer K kullanıcı bulunur ve izlenmeyen ürünlerin puanı bu kullanıcıların puanlarından tahmin edilir. Ürün tabanlı yöntemde (düzeltilmiş) kosinüs benzerliğiyle ürünler arası benzerlik bulunur; gerçek sistemlerde daha çok bu kullanılır.

Bu yöntemlerin ölçek, seyreklik, soğuk başlangıç, hep aynı önerileri yapma, az bilinen ürünler ve ortak hesap gibi sorunları tartışıldı. Seyreklik problemine karşı model tabanlı yöntemler, özellikle bir önceki derste görülen PCA ve SVD ile bağlantılı matris ayrıştırma ve bugün en popüler olan derin öğrenme tabanlı yöntemler anlatıldı. Ders, makine öğrenmesinin büyük veriye kayışı, derin öğrenmenin sınırları ve lisansüstü araştırmada hazır kütüphaneleri birleştirmek yerine asıl problemlere odaklanma tavsiyesiyle kapandı.

Ders Sonrası Notlar

> **Ders sonrası not:** Hoca matris ayrıştırma yaklaşımının "Amazon'un açtığı, 2009-2010 yıllarında galiba bir yarışmada" önerildiğini söyledi ve emin olmadığını belirtti. Söz konusu yarışma Netflix Prize'dır (2006-2009). Yarışma, Netflix'in öneri sisteminin başarısını artıran ekibe ödül veriyordu; 2009'da BellKor's Pragmatic Chaos ekibi kazandı. Matris ayrıştırma yöntemleri bu yarışmayla yaygınlaştı.
>
> **Ders sonrası not:** Hoca tahmin formülünü anlatırken "B'nin o filme verdiği oy eksi B'nin varyansı" dedi, hemen ardından "B'nin genel olarak verdiği oy ortalaması" diye düzeltti. Formülde çıkarılan, B'nin ortalama puanıdır ($\bar{r}_b$), varyansı değil. Paydada da benzerliklerin mutlak değerlerinin toplamı ($\sum|\text{sim}(a,b)|$) kullanılır; böylece negatif benzerlikler paydayı sıfıra yaklaştırmaz.
>
> **Ders sonrası not:** Hoca kullanıcı tabanlı yöntemde "makine öğrenmesi tabanlı bir tahmin yok" dedi. Yöntem aslında K en yakın komşu (k-NN) yönteminin bir uygulamasıdır. Ama ayrı bir model eğitilmez; tahmin her seferinde puan matrisinden hesaplanır, yani tembel öğrenme (lazy learning) gibi çalışır. Bu anlamda "model kurulmuyor" demek doğrudur.
>
> **Ders sonrası not:** Hoca kümeleme tabanlı yöntem için "cluster oluşturmakla ilgisi yok, aslında k-NN" dedi. Literatürde kümeleme tabanlı tavsiye, kullanıcıları ya da ürünleri önce bir kümeleme yöntemiyle (ör. k-means) gruplara ayırıp benzer kişileri ya da ürünleri yalnız aynı küme içinde aramayı da kapsar. Bu, milyonlarca kullanıcıyla karşılaştırma yapma (ölçek) sorununu da hafifletir.
>
> **Ders sonrası not:** Örnek 1'deki iki kullanıcının yalnız iki ortak ürünü vardır; bu kadar az ortak ürünle Pearson katsayısı -1 ya da ona çok yakın çıkar (python ile doğrulandı: ortak ürün ortalamasıyla $-1{,}00$, bütün puanların ortalamasıyla $-0{,}99$).
>
> **Ders sonrası not:** Altyazıda yanlış geçen terimlerin doğru yazımları: "Cosine Simulator" = cosine similarity; "metric spektronizasyon" / "matrix spektronizasyon" = matrix factorization; "kinesi komponent analizi" = principal component analysis (temel bileşen analizi); "K-N, R-S, Neybrut" ve "k-neurisneybrut" = k-nearest neighbor; "Orecibe Vedic" = Recep İvedik; "First Trader" = muhtemelen "first rater" (bir ürünü ilk puanlayan) sorunu; "Osman K" = optimum K; "inertiye" = inertia; "KMİS", "K-Mins" = k-means; "Anad Encoding", "Vanhat Enco Link" = one-hot encoding; "elbo" = elbow.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.